

1. Overview & Architecture

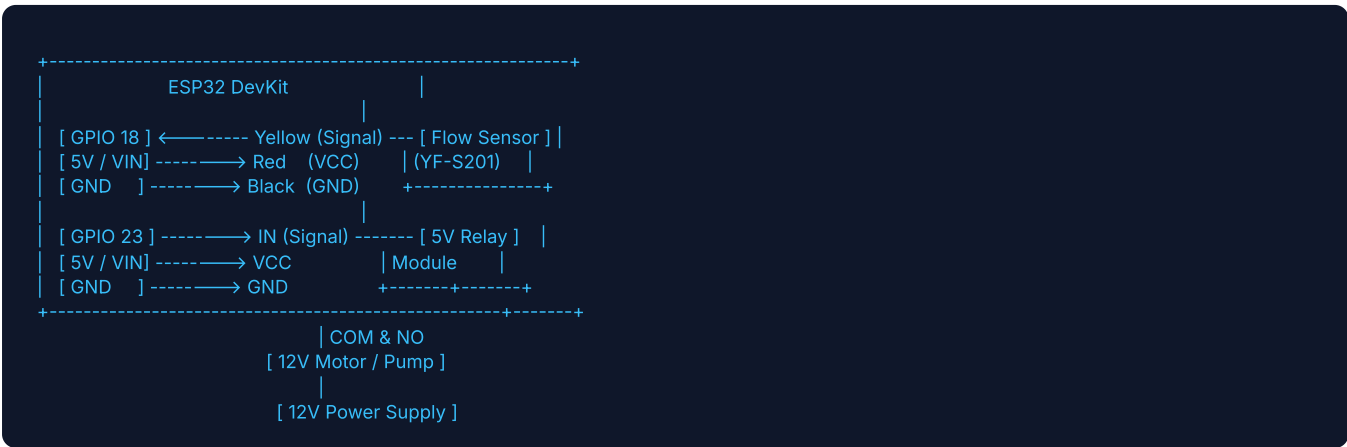
AquaPulse provides real-time groundwater monitoring and automated pay-on-excess enforcement. The physical hardware consists of an **ESP32 microcontroller** connected to a **YF-S201 flow sensor** and a **motor relay valve**. Telemetry is transmitted to an EMQX Cloud MQTT broker, processed by a Flask backend on Render, stored in PostgreSQL, and displayed live on Netlify (aguapulse.netlify.app).

Zero Backend Change Required: The cloud backend and dashboard already adhere to the exact MQTT contract shown below. Flashing the ESP32 with this firmware seamlessly integrates live telemetry without modifying server code.

2. Hardware Bill of Materials (BOM)

Component	Specification	Purpose
Microcontroller	ESP32 NodeMCU / DevKit V1 (ESP-WROOM-32)	Handles Wi-Fi, TLS MQTT telemetry, interrupt counting & relay trigger
Flow Sensor	YF-S201 (Hall-effect, 1/2" inline pipe)	Measures water pulse interrupts and velocity (L/min)
Relay Module	5V 1-Channel Relay (Optocoupler Isolated)	Electromechanically opens/shuts motor power circuit
Water Pump / Motor	12V DC Submersible Pump or Solenoid Valve	Simulates groundwater extraction well pump
Power Supply	12V DC Adapter + 5V Micro-USB Cable	Supplies power to pump and logic power to ESP32

3. Circuit Schematic & Pin Mapping



4. MQTT Cloud Communication Contract

Topic	Direction	Payload Example	Description
gw/device1/usage	ESP32 → Cloud	<code>{"device": "device1", "flow_lpm": 2.4, "litres": 0.2, "total": 132.5}</code>	Published every 5s interval
gw/device1/cmd	Cloud → ESP32	<code>{"relay": "ON"}</code> or <code>{"relay": "OFF"}</code>	Actuates motor valve cutoff / restore

Topic	Direction	Payload Example	Description
gw/device1/status	ESP32 → Cloud	online / offline	MQTT Last Will & Testament

5. ESP32 Arduino Firmware (AquaPulse_ESP32.ino)

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// ===== USER CONFIGURATION =====
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// EMQX Cloud Serverless TLS Broker Details (from backend .env)
const char* MQTT_BROKER = "your-emqx-host.emqxsl.cn";
const int MQTT_PORT = 8883;
const char* MQTT_USER = "your_mqtt_username";
const char* MQTT_PASS = "your_mqtt_password";
const char* DEVICE_ID = "device1";

#define FLOW_SENSOR_PIN 18
#define RELAY_PIN 23
const float CALIBRATION_FACTOR = 7.5; // 7.5 pulses/sec = 1 L/min

// ===== RUNTIME VARIABLES =====
volatile int pulseCount = 0;
float totalLitres = 0.0;
unsigned long lastPublishTime = 0;
bool relayState = true;

WiFiClientSecure tlsClient;
PubSubClient mqttClient(tlsClient);

void IRAM_ATTR pulseCounter() { pulseCount++; }

void callback(char* topic, byte* payload, unsigned int length) {
  String msg = "";
  for (int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.println("[MQTT Command]: " + msg);

  StaticJsonDocument<200> doc;
  if (!deserializeJson(doc, msg) && doc.containsKey("relay")) {
    const char* cmd = doc["relay"];
    if (String(cmd) == "ON") {
      relayState = true;
      digitalWrite(RELAY_PIN, HIGH);
    } else if (String(cmd) == "OFF") {
      relayState = false;
      digitalWrite(RELAY_PIN, LOW);
    }
  }
}

void connectMQTT() {
  while (!mqttClient.connected()) {
    String clientId = "ESP32-" + String(DEVICE_ID);
    String statusTopic = "gw/" + String(DEVICE_ID) + "/status";
    if (mqttClient.connect(clientId.c_str(), MQTT_USER, MQTT_PASS, statusTopic.c_str(), 1, true, "offline")) {
      mqttClient.publish(statusTopic.c_str(), "online", true);
      mqttClient.subscribe(("gw/" + String(DEVICE_ID) + "/cmd").c_str());
    } else { delay(3000); }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH);
  pinMode(FLOW_SENSOR_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), pulseCounter, FALLING);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) { delay(500); }

  tlsClient.setInsecure();
  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(callback);
}

void loop() {
  if (!mqttClient.connected()) connectMQTT();
  mqttClient.loop();
}
```

```

unsigned long now = millis();
if (now - lastPublishTime >= 5000) {
  detachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN));
  float flowRateLpm = ((1000.0 / (now - lastPublishTime)) * pulseCount) / CALIBRATION_FACTOR;
  float deltaLitres = (flowRateLpm / 60.0) * ((now - lastPublishTime) / 1000.0);
  totalLitres += deltaLitres;
  pulseCount = 0;
  lastPublishTime = now;
  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), pulseCounter, FALLING);

  StaticJsonDocument<256> doc;
  doc["device"] = DEVICE_ID;
  doc["flow_lpm"] = serialized(String(flowRateLpm, 2));
  doc["litres"] = serialized(String(deltaLitres, 2));
  doc["total"] = serialized(String(totalLitres, 2));
  char buf[256];
  serializeJson(doc, buf);
  mqttClient.publish(("gw/" + String(DEVICE_ID) + "/usage").c_str(), buf);
}
}

```

6. End-to-End Operational Lifecycle

1. **Physical Pumping:** Water flow spins the sensor turbine, firing interrupts on ESP32 GPIO 18.
2. **Telemetry Cycle:** Every 5 seconds, volume and flow velocity are published via TLS to EMQX.
3. **Live Telemetry:** Flask updates PostgreSQL and the Netlify UI updates gauges in real time.
4. **Threshold Breach & Cutoff:** If usage exceeds the quota limit (e.g. 500L), Flask sends {"relay": "OFF"}. ESP32 trips the relay, instantly halting water extraction.
5. **Payment & Restoration:** The consumer settles the bill via Razorpay on the dashboard. Upon signature verification, Flask sends {"relay": "ON"}, re-energizing the pump.